

PROPUESTA DE ARQUITECTURA PARA CONSOLIDACIÓN DE DATOS

I. Propuesta de arquitectura

A. Campos a extraer de cada fuente

De cada fuente extraería el subconjunto mínimo necesario para cumplir los dos objetivos (consultas SQL operativas y análisis de patrones con clustering/grafos):

F1 - CRM propietario: ID del cliente, nombre, email, teléfono, fecha de nacimiento, género, ciudad, segmento al que pertenece, fecha de alta y fecha de última actualización. Estos campos son necesarios porque enriquecen las transacciones con información demográfica del cliente (útil para SQL operativo) y aportan variables clave para los algoritmos de clustering.

F2 - SQL Server: ID de transacción, ID del cliente, ID del producto, monto, fecha, canal y estatus. Es la base del comportamiento de compra de los clientes para la mitad de productos.

F3 - PostgreSQL: los mismos campos que F2 pero con su propio esquema. Al unificarlos con F2 obtenemos la vista completa de las transacciones de la compañía.

No extraigo tablas de auditoría, logs internos ni campos que no aporten al análisis, para no sobrecargar el pipeline ni las fuentes originales.

B. Retos de extracción y herramientas usadas

Cada fuente presenta retos distintos:

F1 (CRM propietario): al ser una aplicación propia consumida por API REST, no tiene un conector estándar tipo JDBC. Los retos son los límites de la API, la paginación de resultados y que tal vez no tenga marcas de tiempo confiables para identificar registros modificados. Uso AWS Lambda programada por Airflow para llamar al API con paginación, leyendo solo los registros nuevos o modificados desde la última corrida.

F2 (SQL Server): si hiciera lecturas masivas en horario laboral, podría afectar el rendimiento del sistema operativo. Uso AWS DMS en modo CDC, que lee directamente del transaction log de la base de datos. Esto significa que no consulta las tablas activas, solo lee los cambios que ya están registrados en el log, sin bloqueos ni impacto en producción.

F3 (PostgreSQL): mismo reto que F2. Uso AWS DMS aprovechando la replicación lógica de PostgreSQL (WAL), con un slot dedicado solo para el pipeline.

C. Retos por la independencia de modelos de datos

Como las tres fuentes fueron diseñadas de forma independiente, los retos principales son:

Los mismos conceptos tienen nombres distintos en cada base (por ejemplo cliente_id, customer_no, id_cliente).

Los tipos de datos pueden no coincidir (un campo "monto" puede ser decimal en una y numérico en otra).

Las llaves primarias no son compatibles entre fuentes.

Las granularidades pueden ser distintas (una transacción en F2 puede agrupar lo que en F3 sería varias).

Cómo lo resuelvo: dos capas de transformación.

Capa Silver: estandarizo cada fuente por separado (mismos nombres de columnas, mismos tipos, mismos formatos de fecha y moneda), pero mantengo una tabla por fuente.

Capa Gold: unifico las tres fuentes en un modelo común. Creo una tabla de identidad que resuelve qué cliente de F2 o F3 corresponde a qué cliente de F1, usando coincidencias por email o teléfono. El resultado es una vista única "Cliente 360" con todas las transacciones unificadas.

D. Mitigar impacto sobre las fuentes originales

Como las fuentes operan 24/7 y no hay realmente un horario de baja carga garantizado, aplico varias estrategias:

CDC en lugar de extracciones masivas: DMS lee del log de transacciones, no de las tablas en uso. Esto elimina prácticamente el impacto sobre F2 y F3.

Lecturas incrementales en el CRM: Lambda solo trae los registros nuevos o modificados desde la última corrida, en lugar de descargar todo cada vez.

Paginación y control de velocidad: para F1, controlo el ritmo de llamadas al API para no saturarlo.

Carga inicial en horario de menor uso: la primera vez que se sincronizan los datos (la "foto" completa) se hace de madrugada o en fin de semana, que es cuando hay menos actividad. Después solo van cambios incrementales.

E. Etapas del proceso de transformación

Defino tres etapas, siguiendo el patrón medallón (Bronze - Silver - Gold):

1. Zona Raw (Bronze): los datos llegan tal cual salieron de la fuente, sin modificaciones. Sirve para auditoría y para poder reprocesar todo si algo sale mal en las etapas siguientes.
2. Zona Silver: aquí se hace la limpieza por fuente. Se corrigen tipos de datos, se eliminan duplicados, se normalizan textos (mayúsculas, espacios, acentos), se estandarizan fechas y monedas, y se enmascaran datos personales sensibles (email y teléfono). Sigue habiendo una tabla por fuente, pero ya están limpias y estandarizadas.
3. Zona Gold (Curada): aquí se integra todo. Se resuelve la identidad del cliente entre las tres fuentes, se unifican las transacciones de F2 y F3 en una sola tabla con una columna que indica el origen, y se construye el modelo Cliente 360. Esta es la capa que consumen los usuarios finales y los modelos de ciencia de datos.

F. Herramientas para la transformación

AWS Glue (con PySpark) es la herramienta principal en ambas etapas de transformación:

Glue Job de Limpieza: procesa los datos de Raw a Silver. Se ejecuta una vez por fuente, en paralelo.

Glue Job de Modelado: procesa los datos de Silver a Gold. Hace los joins entre fuentes, la resolución de identidad y construye el modelo unificado.

Uso Glue porque es serverless (no hay que mantener servidores), escala automáticamente según el volumen de datos, y se integra de forma nativa con el Glue Data Catalog y Lake Formation para la gobernanza.

Adicionalmente, uso Great Expectations como herramienta de validación de calidad entre cada etapa. Antes de promover datos de Raw a Silver o de Silver a Gold, valida que cumplan reglas básicas (no haya nulos en campos clave, los valores estén en rangos esperados, no haya duplicados en llaves primarias). Si algo falla, se detiene el pipeline y se notifica.

G. Storage por propósito

Uso storages distintos según el tipo de consumo:

Storage	Propósito	Por qué
Amazon S3	(Raw, Silver, Gold)	Almacenar todo el lago de datos. Es barato, escalable y separa el storage del cómputo. Con formato Apache Iceberg gano control de versiones, schema evolution y consultas tipo base de datos sobre archivos.
Amazon Athena	Consultas SQL ad-hoc	de los usuarios operativos. Serverless, sin clúster que mantener, paga por consulta. Es ideal para el usuario que ocasionalmente quiere hacer una consulta SQL.
Amazon Redshift	Dashboards de BI con muchos usuarios concurrentes	Si Power BI tiene muchos usuarios y dashboards refrescándose constantemente, Athena podría quedarse corto. Redshift está optimizado para esa carga.
Amazon SageMaker	Algoritmos de clustering y modelos de ML	Lee los datos en formato Parquet directamente desde S3 Gold. Tiene todo lo necesario para entrenar y desplegar modelos.
Amazon Neptune	Búsquedas en grafos	Es una base de datos especializada en grafos. Permite preguntas tipo "qué clientes están conectados a este otro a través de productos comprados", lo cual sería muy lento en una base relacional.

Cada herramienta resuelve un propósito específico, pero todas leen de la misma fuente única de verdad: S3 Gold. Eso evita duplicar datos y mantener varias "verdades" distintas.

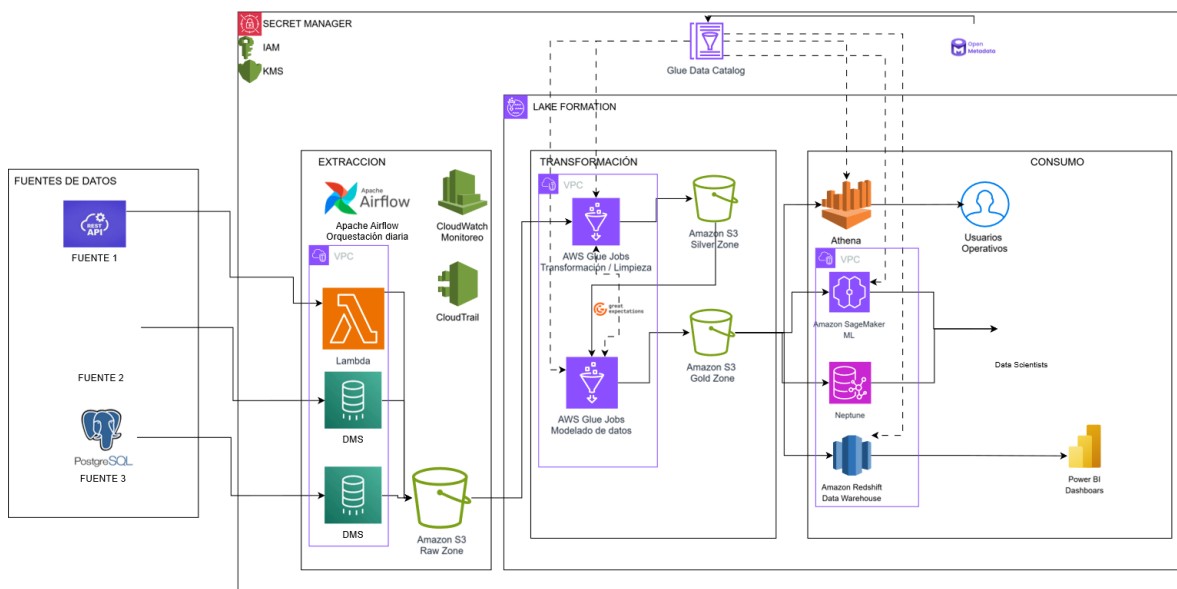
H. Orquestación del pipeline

Uso Apache Airflow (gestionado en AWS con MWAA) para orquestar todo el pipeline. Airflow ejecuta un DAG (un flujo de tareas) diario que sigue este orden:

- Verifica que DMS esté replicando correctamente F2 y F3.
- Ejecuta la Lambda que extrae los datos nuevos del CRM (F1).
- Lanza el Glue Job de Limpieza para procesar Raw → Silver.
- Corre Great Expectations sobre Silver para validar calidad.
- Lanza el Glue Job de Modelado para procesar Silver → Gold.
- Corre Great Expectations sobre Gold.
- Actualiza el grafo en Neptune con los datos nuevos.
- Notifica a SageMaker si hay datos nuevos para reentrenar modelos.

Si alguna etapa falla, Airflow envía una alerta y detiene el pipeline. Elijo Airflow porque es el estándar de la industria, tiene buena interfaz visual para ver qué pasó en cada corrida, y es fácil de versionar en Git.

I. Diagrama



II. Seguridad

A. Cómo aseguro el flujo de datos end-to-end

Aplico seguridad en varias capas, porque ninguna sola es suficiente:

Red: todos los servicios que hacen cómputo (Lambda, DMS, Glue, SageMaker, Neptune, Redshift) corren dentro de una VPC con subredes privadas, sin acceso directo a internet. Para que estos servicios puedan acceder a S3 y Athena sin pasar por internet, uso VPC Endpoints (PrivateLink), que mantienen el tráfico dentro de la red de AWS.

Credenciales: las contraseñas de las tres fuentes (F1, F2, F3) nunca están en el código. Se almacenan en AWS Secrets Manager, que las rota automáticamente cada cierto tiempo. Lambda y DMS las leen en tiempo de ejecución.

Cifrado: los datos están cifrados en reposo usando AWS KMS (en S3, Redshift, Neptune y todos los buckets del lago) y en tránsito usando TLS en todas las conexiones.

Permisos: uso AWS IAM para definir qué servicio puede acceder a qué recurso, siguiendo el principio de mínimo privilegio: cada rol solo tiene los permisos estrictamente necesarios. Además, Lake Formation controla el acceso a nivel del lago, permitiendo restringir tablas e incluso columnas específicas según el rol del usuario (por ejemplo, los data scientists no ven el email del cliente, pero sí su ID enmascarado).

Datos personales (PII): en la capa Silver, los campos sensibles como email y teléfono se enmascaran o tokenizan, para que los analistas trabajen con identificadores anónimos.

Monitoreo y auditoría: CloudWatch captura métricas, logs y alarmas (si un job falla o tarda demasiado, recibo una alerta en Slack o email). CloudTrail registra todas las llamadas a la API de AWS, dejando rastro de quién accedió a qué y cuándo, lo cual es clave para investigar incidentes.

III. Gobernanza de datos

A. Control de metadata, cambios y procesos del pipeline

La gobernanza se divide en tres componentes:

1. Catálogo técnico (qué hay en el lago): uso AWS Glue Data Catalog, que se actualiza automáticamente cada vez que un job de Glue crea o modifica una tabla. Aquí queda registrado qué tablas existen, sus columnas, tipos de datos, particiones y ubicación en S3.

2. Catálogo de negocio y linaje (qué significa cada cosa y de dónde viene): uso OpenMetadata, que se sincroniza con Glue Data Catalog y agrega encima la información de negocio: descripciones funcionales de cada tabla y columna, dueños responsables, glosario de términos, y lo más importante, el linaje completo: de qué fuente original viene cada campo y por qué transformaciones pasó. Cuando alguien

cambia una transformación, OpenMetadata captura la nueva versión y mantiene el histórico.

3. Procesos del pipeline: todo el código del pipeline (DAGs de Airflow, jobs de Glue, validaciones de Great Expectations) vive en Git, con control de versiones completo. Cada cambio queda registrado: quién lo hizo, cuándo y por qué. Cuando se despliega a producción, Airflow guarda el historial de cada ejecución (qué corrió, cuándo, cuánto duró, si tuvo éxito o falló, cuántos registros procesó).

Almacenamiento de la metadata:

El catálogo técnico vive en Glue Data Catalog (servicio gestionado de AWS).

El catálogo de negocio y el linaje viven en OpenMetadata, respaldado por una base de datos PostgreSQL.

Las definiciones del pipeline viven en Git.

El historial de ejecuciones vive en Airflow y en CloudWatch (logs centralizados).

Con esto se cubre todo el ciclo: qué datos existen, qué significan, de dónde vienen, qué se les hizo, quién los toca y cuándo.